The CMDB is the database while the Common Service Data Model (CSDM) is the data model. The data model helps determine where to record items in the CMDB, and what relationships are important to drive business value.

Configuration Item (CI)

Physical and logical components of an infrastructure that needs to be managed to deliver a product or service and are currently, or soon will be, under configuration management. For example, a network device, a server, an application, a **delivery truck**, or service.

Attribute

Information that further describes a CI such as a name, serial number, manufacturer, operating system.

Relationship Type

Include the relationships between configuration items. For example, a web application might read data from a specific database, which in turn might depend on a piece of underlying hardware.

Class

Describes a table that contains and represents a specific type or group of CIs that share common attributes such as a Windows Server, Linux Server, Printer, Virtual Machine, **Vehicle**, **Animal** etc.

Configuration Management Database (CMDB)

A database used to store configuration records throughout their Lifecycle. **The Configuration Management System maintains one or more CMDBs**, and each CMDB stores attributes of CIs, and Relationships with other CIs.

Base Table

The core Configuration Item [**cmdb_ci]** table, which stores the basic attributes of all the CIs. **All configuration item classes extend from this table** including all hardware and applications.

CI **Reclassification**

Is a CI whose **class** has been **upgraded, downgraded, or switched**. An example of an upgraded CI is a Server record that was upgraded from the Server [cmdb_ci_server] class to the Windows Server [cmdb_ci_win_server] class.

Parent and Child Class

A table that extends another table is called a child class, and the table it extends is the parent class. A table can be both a parent and child class both extending and providing extensions for other tables. For example, the Server table extends the Computer table and the Windows Server table extends the Server table, thus making the Server table both a Parent and Child class.

Asset vs. CI

On the Now platform, when creating a hardware asset, a corresponding CI will be automatically created or when a CI is discovered for the first time and inserted into the CMDB, a corresponding asset record will be automatically created. The asset and the CI is connected through out the CI/Asset lifecycle.

Asset:

- Often starts during the procurement process, but may be created when a discovery tool finds the CI for the first time
- Is part of the financial lifecycle

Configuration Item (CI):

- Often starts when a discovery tool finds the CI for the first time, but may be created during the procurement process
- Is part of the technical operations

Common Services Data Model (CSDM)

Represents a standard and shared set of **service related definitions** across ServiceNow products and the NOW platform that enables and supports true service level reporting while providing prescriptive guidance on service modeling within the CMDB. These service-related definitions span the ServiceNow® product portfolio and the Now Platform®.

Application Service

A set of interconnected applications and hosts which are configured to **offer a service to the organization**. Application services can be internal, like an organization email system

or customer-facing, like an organization website.

Application

Applications that comprise an application service as configuration items (CIs). The various Applications and the relationships between them, that comprise an application service, are stored in the CMDB. (i.e. Tomcat, Apache, Microsoft SQL, Oracle, IIS).

DevOps

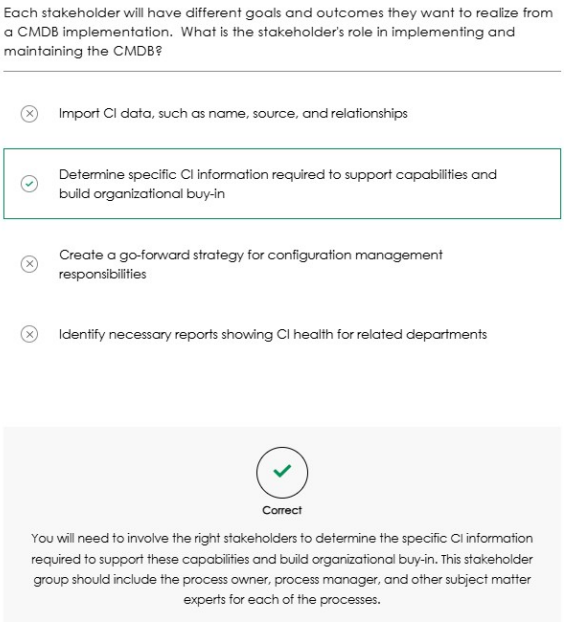
Consists of developers who build applications and the operations groups who deploy and maintain them within an enterprise. In a DevOps environment, these two groups are combined into one in order to work more effectively together to accelerate and improve service delivery.

Principal Class

Principle Classes can be configured **to limit the CI classes that display** in the Configuration Item fields on incident, problem, and change records.

Logical CI

Do not take up space. They perform specific functions requiring physical components in order to operate. Examples include Service Offering, Service, Business Application which are manually entered into the CMDB.



Examples:

1 **Business Application**

- **Logical CI:** “Microsoft Teams” (the collaboration/chat app used by employees)

- **Physical CIs it depends on:**
    - Cloud servers hosting Teams
    - Network infrastructure (routers, switches)
    - Databases storing user info and messages

---

2 2️⃣ Service

- **Logical CI:** "Corporate Email Service"
- **Physical CIs it depends on:**
    - Exchange Server or Office 365 backend
    - Mail servers
    - Storage for email data
    - Network connections for email delivery

---

3 3️⃣ Service Offering

- **Logical CI:** "VPN Access for Remote Employees"
- **Physical CIs it depends on:**
    - VPN servers
    - Firewalls
    - Network links to the corporate office

---

4 4️⃣ Other Examples

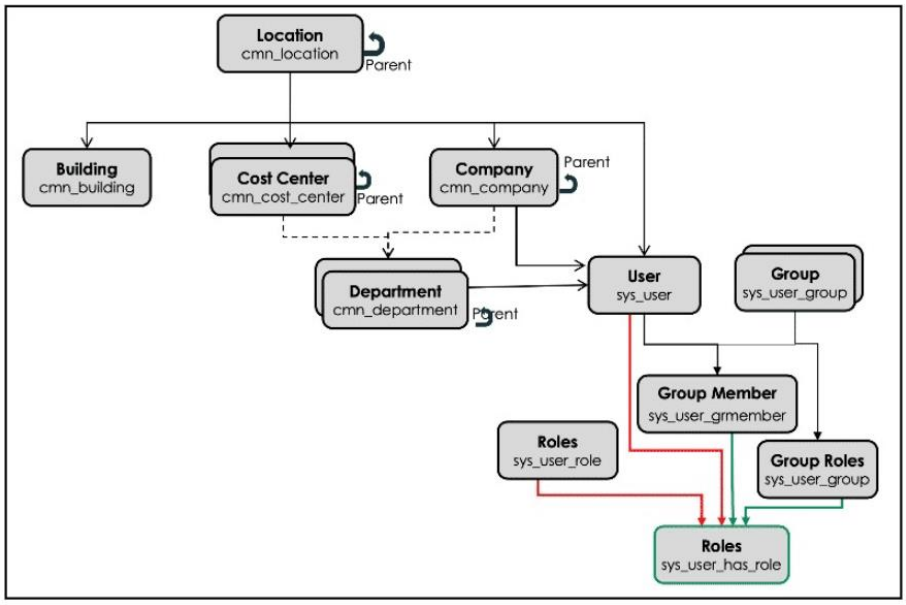
- **Logical CI:** "HR Payroll System"
    - Depends on: Payroll application servers, database servers, authentication servers.
- **Logical CI:** "Customer-Facing Website"
    - Depends on: Web servers, application servers, load balancers, content delivery network.

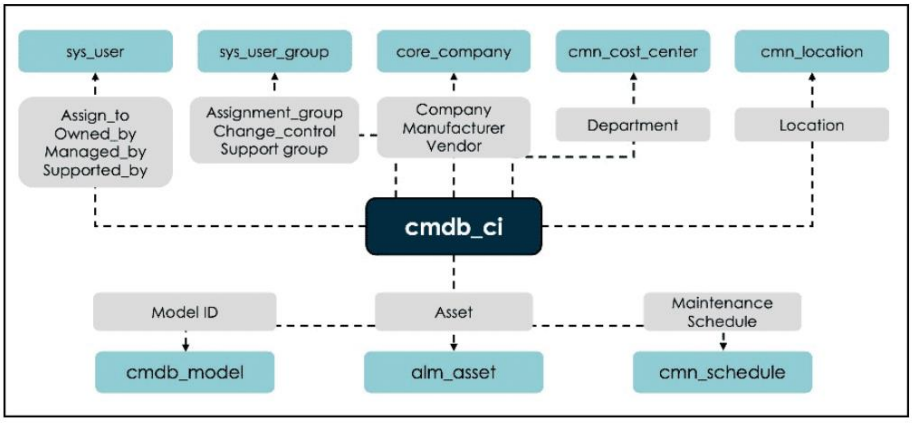
💡 **Tip:** A simple way to think of it:

Logical CI = What the business sees / uses
Physical CI = What actually runs it behind the scenes

Foundation data:

Not part of the CMDB, but referenced In the CI records.



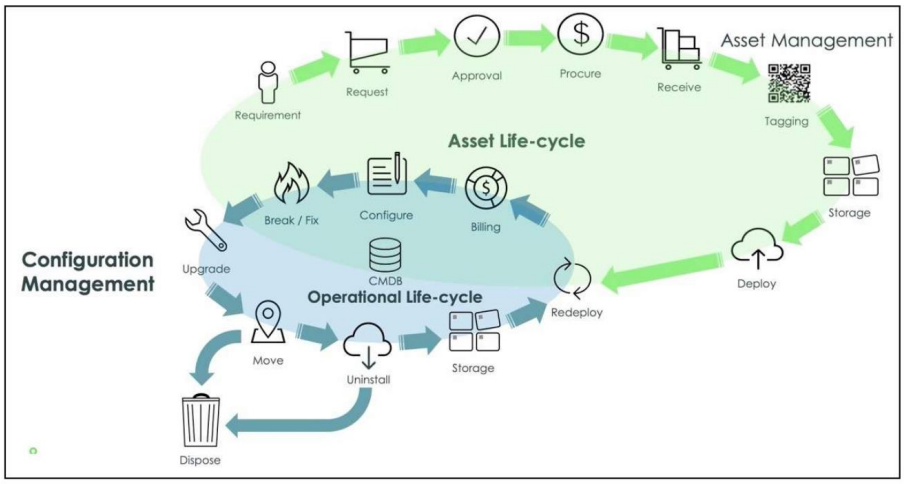


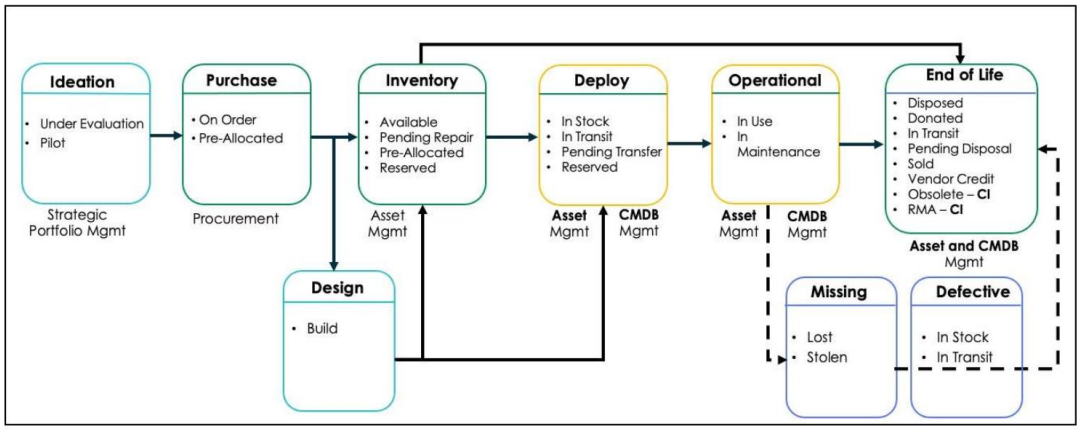
One of the most important and challenging aspects of CMDB design is selecting the appropriate CI classes, levels, attributes, and relationships to be used as part of a CMDB implementation. **During a CMDB health check, ServiceNow examines the number of custom classes and attributes created and assesses whether these custom attributes are established at the correct level. ServiceNow provides a free store application called CMDB CI Class Models that can be installed on a ServiceNow instance to obtain a wide range of additional classes that are not provided in a base instance.**

Automated discovery tools by SN: ServiceNow Discovery, Service Mapping, or Agent Client Collector

Configuration and Asset Management Lifecycles

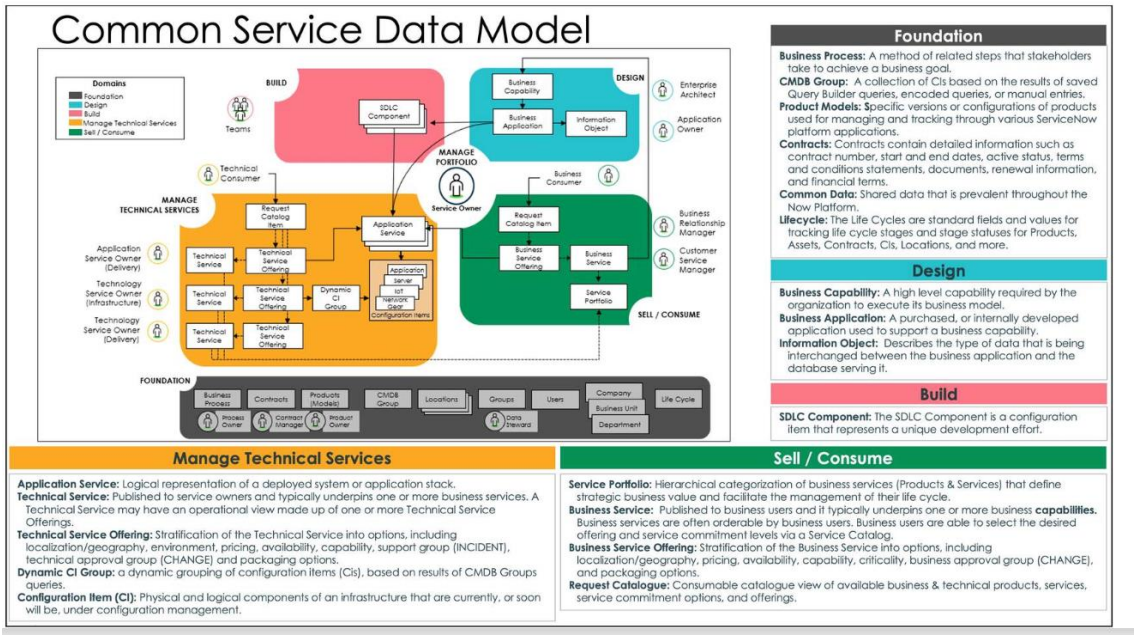
An asset becomes a CI (Configuration Item) when it is deployed and remains a CI throughout its lifecycle until it is returned to stock or disposed of. As illustrated in the diagram, Configuration Management and IT Asset Management are distinct, yet complementary practices.





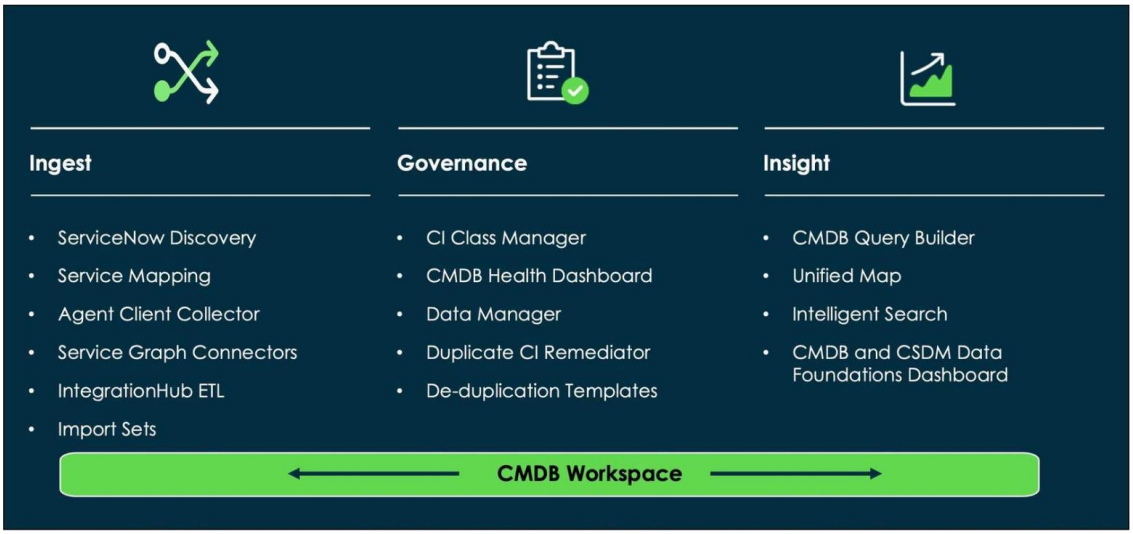
Change Control Board (CCB)

Form a Configuration Control Board (CCB). The CCB is a team of people that ensures your CMDB delivers continuous value. This team sets direction and makes decisions on what is needed to ensure your CMDB can support your organization's strategic initiatives and determine if the CMDB is ready for utilization.



To operationalize and maintain the health of the CMDB, ServiceNow has created three of pillars to success. At the highest level each pillar focuses on a key customer pain point.

- **First is Ingestion of CMDB data**: This is how the CMDB is populated and the data is identified and reconciled. Automate as much as possible the flow of data into the CMDB at a frequency that supports your stated use cases
- **Second is Governance of the data:** Getting the data in is just the beginning of your CMDB journey. Next is the management and upkeep of the data to maintain a healthy CMDB and gain value from the data.
- **Third is Insight into the data in the CMDB**: How do you visualize and utilize this data across the platform?



ServiceNow CMDB Data Population & Discovery Methods (Ingestion)

ServiceNow provides multiple approaches to populate and maintain the CMDB, ranging from automated discovery to integrations and data imports. Each method serves a specific purpose and has licensing and configuration considerations.

1. **ServiceNow Discovery (Horizontal / Agentless, IP-based discovery)**

- Automatically discovers infrastructure (servers, network devices, applications).
- Populates the CMDB and maps dependencies between components.
- **License**: Paid (ITOM Visibility).
- **Note**: Discovered data should be enriched with non-discoverable attributes (e.g., support group, managed by).
- **Schedule based; not real-time.**
- It uses probes and sensors to determine the device type.

MID Server:

- The **MID Server (Management, Instrumentation, and Discovery Server)** is deployed inside the organization's **trusted network**.
- It acts as a **secure bridge** between the ServiceNow instance and internal systems that are not directly accessible due to firewalls or network restrictions.
- All communication between ServiceNow and the MID Server is **encrypted**.
- ServiceNow sends **probes** to the MID Server; the MID Server executes them and returns results, which **sensors** use to update the **CMDB**.
- Discovery is **agentless** and uses standard protocols:
    - **WMI** for Windows
    - **SSH** for Linux/Unix
    - **SNMP** for network devices
- The MID Server ensures **secure, scalable Discovery** without exposing internal systems to the internet.

✓ Key takeaway: *ServiceNow never directly connects to internal devices—everything goes through the MID Server.*

Discovery Schedules:

- **Discovery Schedules** define *when*, *what*, and *where* Discovery runs.
- Used for **horizontal Discovery** (network-based scanning).
- Specify:

    - **IP ranges / scope**
    - **Schedule timing**
    - **Assigned MID Server(s)**
- Can target **on-prem environments** or **cloud service accounts**.
- Schedules trigger Discovery using **probes, sensors, and patterns** to identify and update **CIs in the CMDB**.

Discovery Admin Workspace (Schedules Page)

- Central hub to **monitor and manage Discovery operations**.
- Provides **real-time visibility** into Discovery activity and performance.
- Key reports include:
    - **Schedules by location** (map)
    - **Total schedules** (last 24 hours)
    - **Errors by category**
    - **Total Discovery attempts** (completed vs canceled)

✓ Key takeaway: *Discovery Schedules control horizontal Discovery execution and are monitored through the Discovery Admin Workspace.*

Types of Discovery:

- **Network Discovery**
    - Identifies **internal IP address ranges**.
    - Not required if IP ranges are already known.
- **CI Discovery**
    - Discovers **devices, servers, and applications**.
    - **Most commonly used** Discovery type.
- **Cloud Discovery**
    - Discovers **AWS and Azure resources**.
    - Uses cloud credentials and APIs.
- **Serverless Discovery**
    - Discovers **applications without discovering the host first**.
    - **Pattern-based** and does not rely on traditional host discovery.

✓ Key takeaway: *All are forms of horizontal Discovery, each targeting different environments and use cases.*

Limiting Discovery Data:

- **Improves data accuracy**
  - Avoids populating the CMDB **with low-priority** or irrelevant data.
  - Establishes a strong foundation with high-quality CIs.
- **Prevents CMDB data overload**
  - Reduces clutter caused by unnecessary Discovery results.
  - Keeps the CMDB clean and maintainable.
- **Faster initial deployment**
  - Fewer devices scanned → quicker Discovery cycles.
  - Enables a usable CMDB early, with gradual expansion later.
- **Simplifies CMDB management**
  - Focuses on key CI classes (e.g., servers, critical applications).
  - Ensures data is immediately relevant to business needs.
- **Better stakeholder buy-in**
  - Quick wins with accurate data build trust.
  - Encourages support for broader Discovery adoption later.

✓ Key takeaway: *Start small, discover what matters first, then expand Discovery incrementally.*

Discovery Ranges:

Discovery ranges define which IP addresses ServiceNow Discovery scans.

- You can **include or exclude specific IP ranges** in a Discovery schedule.
- **Broad Ranges (Subnets)**
- Example:
  - 192.168.0.0/24
  - 10.11.128.192/26
- Includes **network and broadcast addresses**.
- Scans a **large number of IPs**, which may cause:
  - Data overload
  - Redundant or irrelevant CIs
  - Discovery errors
- **Not recommended** during initial implementation.
- Best practice: **Do not exceed /16** (~65,000 IPs); split into smaller ranges if needed.

Limited Ranges (Preferred Initially)

- Example:
  - 192.168.0.1 – 192.168.0.254
  - 10.11.128.193 – 10.11.128.254
- Excludes network and broadcast addresses.
- Targets **specific, known devices**.
- Improves **data quality**, **performance**, and **troubleshooting**.
- Supports **clean and manageable CMDB growth**.

✓ Key takeaway: *Start with small, targeted IP ranges and expand Discovery gradually to avoid CMDB data overload.*

Discovery Configuration Console:

- Used to **control what Discovery detects** (devices, applications, software, and CIs).
- Allows **including or excluding specific CI types** from Discovery.
- Excluding a CI **disables the related probe or classifier**.
- Helps CMDB teams focus on **principal CI classes** only.
- Reduces **noise, unnecessary data, and CMDB clutter**.
- Access path: **Discovery Definition → Configuration Console**

✓ Key takeaway: *Use the Configuration Console to limit Discovery to only the CIs you want to manage.*

Discoverable Data:

- **Discoverable data** is the hardware, software, and network information collected by ServiceNow Discovery.
- Common examples include:
  - **Hardware**: CPU, memory, storage
  - **Software**: operating systems, installed applications
  - **Network**: IP address, MAC address, topology
  - **Cloud resources**: AWS, Azure, Google Cloud
- Discovery runs on a **scheduled basis** to keep the **CMDB up to date** with infrastructure changes.
- **Discovery Source Field**
- Newly discovered CIs are created with **Discovery source = ServiceNow**.

- If a CI already exists (e.g., created via **Asset Management**), the source may initially be **SNAssetManagement**.
- Once discovered, the **Discovery source is updated to ServiceNow**.

✓ *The Discovery source always reflects the last **authoritative** discovery method.*

Non-Discoverable Data:

- **Non-discoverable data** cannot be collected by automated tools like Discovery, Service Mapping, or Service Graph Connectors.
- Must be **manually maintained or populated via automation** after Discovery.
- Used to support **ITSM processes** such as Incident, Change, and Problem Management.

Common Non-Discoverable Fields

- **Assigned to**
- **Owned by**
- **Support group**
- **Change group**
- **Managed by group**
- **Location**
- Accurate group data is critical for:
    - Proper **incident routing**
    - Efficient **change execution**
    - Operational accountability

✓ Key takeaway: *Discovery populates technical CI data; CMDB admins must enrich CIs with non-discoverable ownership and group information.*

2. Service Mapping (Top-Down / ML-Powered, Tag-based discovery)

- Builds end-to-end service maps showing how business services are supported by IT components.
- Dynamically updates relationships as changes occur.
- **License**: Paid (ITOM Visibility).
- **Note**: Follow CSDM guidelines, define service owners, and relate services to business applications.

Business Value:

- **Bridges the gap between IT Operations and business users**.

- Aligns IT service delivery with business outcomes, not just infrastructure.
- Supports ITSM processes: Incident, Problem, and Change Management.

Key Concepts:

- **Application Service:** Logical representation of a deployed system, application, or product stack.
    - Operational CI used in **ITSM**/ITOM practices.
    - Can be environment-specific (Dev, Test, QA, Prod) or region-specific (AMS, EMEA, APJ).
    - Stored in cmdb_ci_service_auto or sub-classes.
- **Service Map:** Shows which CIs support a service, including applications, servers, middleware, and networks.
    - Helps identify service impact when components fail.

Types of Application Services

1. Top-Down Mapping

- **Pattern-Based:**
    - **Best for:** Mission-critical apps.
    - **Level:** Surgical/Process-to-process.
    - **Key:** Most **Granular**.
- **ML Connection Suggestions:**
    - **Best for:** Homegrown & custom COTS.
    - **Level:** Traffic-based/Fingerprinting.
    - **Key:** Fastest **Time-to-Value**.
- *Note: These two can be used together or individually.*

---

2. Tag-Based Mapping

- **Best for:** Cloud-native, Containers, and VMs.
- **Effort: Medium touch** (requires defining tag categories/families).
- **Key:** Visualizes resource stacks across cloud infrastructure.

---

3. Dynamic CI Group

- **Best for:** Small application services.
- **Mechanism:** Uses CI list filters or CMDB queries.
- **Effort: High touch** (requires manual query building).